Programmable Sovereignty Over Reversible Action

Anonymous for external review

Abstract

Constitutional admission has been described as the test a polity applies to incoming receipts at the boundary of its own log. The amendment side of that lifecycle is conditioned at the type level on a backward-refinement witness, so a candidate constitution cannot be enacted in the well-typed runtime path unless the proof of refinement is supplied as data. The response side has been left to the admission predicate alone. The construction described here extends the discipline of the amendment side to enforcement actions whose effects are reversible by an inverse executor. An executive action is a type whose constructor refuses to build a witness without a positive time-to-live duration and an action class tag; reversible variants admit an optional rollback receipt that closes the action chain; destructive variants reduce admission to bilateral predicate intersection between a device polity and an operator polity. The load-bearing claim, mechanized in a paper-local Lean model checked against the substrate's syntactic treaty model, is a composition theorem stating that a chain of time-bounded amendments, each carrying a per-step refinement witness on an explicit finite admission domain, preserves the baseline constitution on that domain at every instant under the auto-reversion semantics of the TTL window. The empirical anchor is an endpoint substrate in which four reversible action variants have real OS executors with rollback constructors, with the file-class variants gated on content-hash parity against the recorded forward artifact and the process- and network-class variants gated on substrate-recorded inverse witnesses; the rollback inversion is checked at the receipt level by the same canonical-byte machinery that signs admission, and the bilateral-destructive variant is reported as a deployment obligation rather than a measurement.

1 Introduction

A constitutional polity has two halves of authority. The first half is the admission predicate that decides whether an incoming receipt belongs to the polity's history; the second half is the positive enforcement act the polity issues when its constitution requires response to an admitted fact. The amendment lifecycle has been formalized on the first half: a candidate constitution becomes the active constitution only when the amendment witness carries a Lean term establishing that every receipt admitted by the new predicates was also admitted by the old. The response side has been left to the admission predicate, which means an enforcement act has had no positive type-level discipline of its own: the only thing the type system enforced was that the act admitted, not that the act terminated, reverted, or carried a counterparty's signature.

The reversible-action discipline extends the type-level rule from amendment admission to enforcement response, for the class of actions whose effects are reversible by an inverse executor. A reversible executive action is a type whose constructor refuses to build a witness without a positive time-to-live duration and an action class tag; the inverse executor produces a rollback receipt that closes the action chain by referencing the original execution receipt and the executor key that signed the rollback. A destructive action, defined by the absence of an inverse executor, reduces admission to bilateral predicate intersection between a device polity that hosts the action and an operator

polity that authorizes it; the reduction specializes the prior treaty intersection theorem [6] to the destructive class.

The load-bearing composition claim relates the response side to the amendment side. A TTL-bounded amendment is a constitutional delta paired with a positive TTL duration: the new constitution is active inside the window, and the old constitution returns when the window expires. Such amendments form a chain when issued sequentially. Each chain step carries the same per-step backward-refinement witness the amendment lifecycle already requires. The composition theorem states that for any instant during the chain's lifetime, the active constitution refines the baseline at that instant, regardless of whether the instant falls inside or outside any given amendment's TTL window. The auto-reversion at TTL expiry is exactly the structural mechanism that keeps the chain's trajectory pinned to the baseline rather than drifting predicate-by- predicate into a ratchet.

The contributions are four artifacts that follow the same falsifiability discipline as the prior substrate [6].

- A formal model relating the executive-action type, its TTL invariant, its optional rollback receipt, and the TTL-bounded amendment chain (Section 4).
- A load-bearing composition theorem `ttl_bounded_amendment_chain_preserves_baseline` and a supporting rollback-receipt admission theorem `rollback_admission_composes_with_refinement`, both mechanized in a paper-local Lean model on an explicit finite admission domain, with two definitional bridges retained to anchor the type-level discipline (Section 4).
- A proposed endpoint substrate instance in which four reversible action variants would have inverse executors with content-hash-gated rollback constructors and a four-receipt grammar (request, execution, rollback, acknowledgement) records each step of the action chain (Section 3).
- An evaluation plan that names the reversible-path harnesses, fixture corpora, destructive class, missing TTL scheduler, and stub endpoint sensor as gaps rather than reporting unshipped measurements (Section 6).

The sovereignty claim is bounded. Sovereignty over reversible action covers the polity's receipt log, not the physical world; a rollback receipt is a statement about that log referencing an inverse executor's signed assertion that the OS operation completed. The receipt-log discipline is the falsifiable part; the physical reversibility is an empirical claim about specific executor pairs (rename and inverse rename, signal stop and signal continue) that the receipt schema records without itself proving.

2 Background

Compensating transactions and the saga pattern give the long-running analog to atomic transactions for workflows whose commit boundary is weaker than serializable schedules [1]. A saga is a sequence of forward steps each paired with a compensating step that semantically undoes the forward step. Workflow nets extend the model with timed arcs and soundness criteria that decide whether every reachable marking has a path to a final marking. The category of effects reversible by inverse executors (file rename, signal stop / continue, network policy reload) is a strict subclass of the compensating-action literature; the reversible-action substrate inherits the saga vocabulary without the workflow-net soundness obligations, which would require a timed transition system the substrate does not provide.

Capability revocation in operating systems has the closest structural analog. EROS, seL4, and Capsicum each treat revocation as a typed operation on a capability token: the capability becomes unusable after revocation, and the revocation event is itself a checkable artifact. Bounded executive action shares the revocation semantics and adds a constitutional admission layer above it: the act of revoking is not just an OS operation but a receipt whose admission predicates include the bilateral-cosignature requirement when the action class is destructive. The OS-capability literature solved revocation at the machine boundary; the substrate lifts the discipline to the cross- organizational boundary where the receipt graph crosses kernels.

Endpoint detection and response engines have shipped TTL fields and manual rollback workflows for a decade. Vendor schemas record an action identifier, a target host, an action class, a TTL duration, and a rollback handle; auditors inspect the rollback handle by hand when something goes wrong. No deployed engine reduces TTL enforcement to a typed invariant, and no engine documents a rollback receipt admission predicate distinct from the original execution receipt's admission predicate. The contribution is not a new engine; it is a typed formulation of the four-receipt grammar (request, execution, rollback, acknowledgement) and an admission predicate stack that the existing engines could in principle adopt.

The prior substrate [6] supplies four primitives the reversible-action discipline depends on. The treaty intersection theorem `treaty_admission_iff_predicate_intersection` gives the bilateral reduction. The ladder-floor stability theorem `treaty_admission_stable_under_ladder_floor` gives the receipt- backed floor under which destructive actions are inadmissible. The amendment refinement bridge `amendment_admissible_iff_backward_refinement` gives the type-level invariant on enactment. The essential-predicate chain theorem `essential_preserved_chain` gives the trajectory-invariant composition over amendment sequences. The composition theorem added here is the response-side analog of the fourth primitive: TTL-bounded amendments compose with the chain machinery the prior substrate already established.

The Anthropic alignment-evaluation literature on alignment-faking and situational-awareness is a non-substrate motivation. A scheming agent that behaves under evaluation and differently in deployment leaves no checkable artifact unless the runtime emits one. A typed reversible- action discipline binds the agent's destructive actions to a bilateral admission predicate the deploying organization participates in, so the artifact cannot be retroactively denied. The substrate does not solve scheming; it makes the consequence of an agent's destructive action recordable in the same canonical form the prior substrate's receipts [6] already take.

3 The Reversible Action Substrate

A reversible action substrate consists of an action taxonomy, a four- receipt grammar, an inverse executor for each reversible action, and a TTL-bounded amendment chain that conditions the active constitution on the instant the receipt is admitted. The taxonomy and the grammar are typed; the inverse executors are OS-level operations; the chain is the substrate-level model that connects the response side to the amendment side of the prior substrate [6].

Action taxonomy. The action taxonomy has twelve variants partitioned into three classes. Five variants (allow, observe, warn, alert, block) are verdict tags that do not dispatch to an executor; they are observed by the admission predicate and recorded by the receipt grammar without OS- level side effects. Four variants (file quarantine, persistence disable, process-tree suspend, egress restriction) are reversible: each has a forward executor and an inverse executor, and the rollback receipt is admissible when the inverse executor's signed completion matches the forward executor's recorded

Variant	Class	Rollback witness	TTL
File quarantine	reversible	content-hash-gated restore	auto-revert
Persistence disable	reversible	content-hash-gated re-enable	auto-revert
Process-tree suspend	reversible	SIGCONT (substrate-issued capability)	auto-revert
Egress restriction	reversible	capability-revoke (substrate-issued)	auto-revert
Process-tree terminate	destructive	absent; bilateral cosignature at admission	no-revert

Figure 1: The five operational action variants and their rollback-witness shapes. Four reversible variants admit content-hash-gated or capability-token rollback witnesses constructible at the moment of admission; the destructive variant reduces admission to bilateral predicate intersection between a device polity and an operator polity, since no rollback witness is constructible.

effect. Three variants (process-tree terminate, network isolate, grant revoke) are destructive: each has a forward executor without an inverse, and admission reduces to bilateral predicate intersection between a device polity and an operator polity.

The partition is structural rather than operational. A reversible variant is one for which an inverse executor exists whose effect, when applied to the post-effect state, returns the pre-effect state up to the substrate's notion of equivalence. File rename is the canonical instance: a forward rename moves a file to a quarantine root, the inverse rename moves it back to its original path, and the content- hash gate on the inverse rejects the operation if the original path has been overwritten in the interim. Process-tree suspend via SIGSTOP and its inverse SIGCONT is a second instance: the inverse is valid against the surviving pid set, with the substrate recording which pids in the original target list have since exited. Figure 1 summarizes the five operational variants and the shape of the rollback witness each admits.

Four-receipt grammar. Each action chain emits up to four receipt families. A request receipt records the action plan: action identifier, action class, target root node, dry-run flag, TTL duration, rollback reference, reason, issued- at timestamp, expires-at timestamp, and node and edge counts of the target subgraph. An execution receipt records the executor's completion: execution identifier, action identifier, started-at and completed-at timestamps, an evidence bundle, the actor, and the effects produced (forward effects only). A rollback receipt closes the chain when the inverse executor runs: rollback identifier, execution identifier, action identifier, rolled-back-at timestamp, executor key, and the inverse effect set with content-hash parity check against the forward effect set. An acknowledgement receipt records the operator's attestation that the chain has been observed: acknowledgement identifier, execution identifier, acknowledged-by, optional note, and optional control-correlation token carrying the acknowledgement-token hash.

The receipt grammar is canonical-byte stable. Each receipt body is serialized under the RFC 8785 canonical JSON discipline the prior substrate [6] already requires, signed under the polity's Ed25519 keypair, and appended to a per-family ledger. Cross-family references (rollback's execution identifier, acknowledgement's execution identifier, control-correlation's response-action identifier) are checked at verification time by the verifier-owned ledger lookup, not by trust in the signing kernel.

Bilateral envelopes on destructive variants. A destructive action's admission requires a bilateral DSSE envelope over the canonical request body. The device-polity signature and the operator-polity signature must be drawn from independent keys; the strict bilateral DSSE verifier carried by the prior substrate [6] (`crates/trust/chio-federation/src/bilateral_dsse.rs`) rejects en-

velopes with reused signer keys, wrong predicate types, and noncanonical payloads. The reduction `destructive_action_requires_bilateral_admission` in Section 4 specializes the treaty intersection theorem to the destructive subclass of the action taxonomy.

TTL-bounded amendment chain. A TTL-bounded amendment is a constitutional delta paired with a positive TTL duration. The pre-amendment constitution is the substrate's baseline; the post-amendment constitution is the constitution active inside the TTL window; the auto-reversion at TTL expiry returns the baseline. The chain is the sequence of such amendments issued during the substrate's lifetime, indexed by issued-at timestamp.

The chain is the response-side analog of the prior substrate's amendment trajectory [6]. The chain machinery `essential_preserved_chain` proves that essential admission is preserved across an amendment sequence whose every step carries a preservation witness. The TTL-bounded variant adds the auto-reversion: each step's effect is bounded by the TTL, and the baseline is the substrate-level invariant the chain returns to whenever the topmost window expires.

The substrate does not assume real-time semantics. The substrate treats time as a monotonic integer counter, and the TTL is a duration in the same counter's units. A real-time scheduler (deployed separately) advances the counter and triggers expiry callbacks; the substrate model abstracts the scheduler as an axiom in Section 9.

4 Formal Model

The model abstracts away the OS executor, the canonical JSON encoder, and the receipt ledger; those lower layers are substrate-level obligations outside the model, discharged by code citation rather than by mechanized refinement. The institutional core that remains is the executive-action type, its TTL invariant, the rollback receipt, the TTL-bounded amendment chain, and the composition theorem connecting the response side to the parent paper's amendment side.

Executive action. An executive action is a structure that cannot be constructed without a positive TTL witness:

$$\mathsf{ExecutiveAction} = (\mathit{rid},\ t_{\text{issued}},\ \mathit{ttl},\ \mathit{ttlPos},\ \mathit{kind},\ \mathit{rb})$$

where rid is the receipt identifier, t_{issued} is the issued-at instant, ttl is the duration in counter units, ttlPos is the field-level proof $0 < \mathit{ttl}$, kind is the action class tag, and rb is an optional rollback receipt. The expiry $t_{\text{expires}} = t_{\text{issued}} + \mathit{ttl}$ is derived structurally.

The action is closed at instant t iff a rollback receipt exists with $\mathit{rolledBackAt} \le t$ or the TTL window has elapsed by t. This is recorded as the definitional bridge `rollback_closes_or_ttl_window_active`, which discharges by a case split on the TTL window. The bridge is not the headline theorem; it documents the relationship between the type-level fields and the propositional closure claim, in the same role `amendment_admissible_iff_backward_refinement` plays for the amendment side of the parent paper.

TTL-bounded amendment. A TTL-bounded amendment A is a constitutional delta δ paired with an issued-at instant t_{issued} and a positive TTL duration ttl. The active constitution at instant t is

$$\mathsf{activeAt}(A, t) = \begin{cases} \delta.\mathit{old} & \text{if } t_{\text{issued}} + \mathit{ttl} \le t, \\ \delta.\mathit{new} & \text{otherwise.} \end{cases}$$

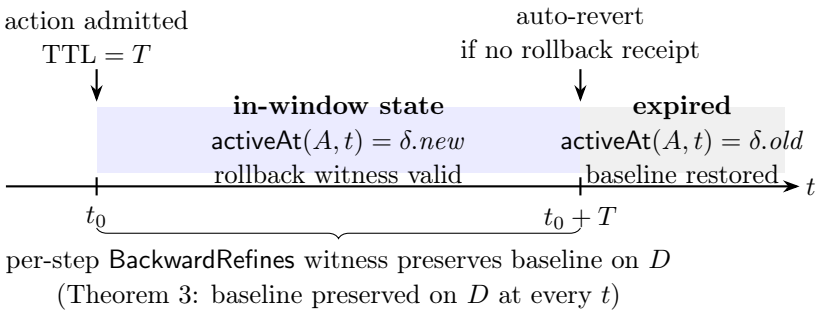

Figure 2: TTL-window evolution. An admitted action is bound to a positive TTL T at admission time t_0. Within the window $[t_0, t_0 + T]$, a per-step refinement witness preserves the baseline. At $t_0 + T$, auto-reversion fires if no explicit rollback receipt has closed the chain. The composition theorem (Theorem 3) states that the baseline is preserved on the finite admission domain D at every instant.

The case split is a substrate-level abstraction of the TTL scheduler's behavior. The scheduler advances the counter and reverts the amendment at t_{expires}; the model represents this as a conditional on t. Figure 2 illustrates the evolution of an admitted amendment across the window boundary.

The composition theorem. The headline composition theorem `ttl_bounded_amendment_chain_preserves_baseline` states that a chain of TTL-bounded amendments, each carrying two witnesses over one explicit finite admission domain D (one that the pre-amendment constitution backward-refines a fixed baseline on D, and one per-step witness that the post-amendment constitution backward-refines the pre-amendment constitution on D), preserves the baseline on D at every instant:

$$\forall t,\, \forall A \in \mathit{chain},\ \mathsf{BackwardRefines}\big(\mathsf{activeAt}(A, t),\, \mathit{baseline},\, D\big).$$

Here $\mathsf{BackwardRefines}(c', c, D)$ is the substrate's domain-scoped refinement: every admission view in D that c' admits is admitted by c. The statement is finite-domain, matching the witness the substrate's amendment check carries; it says nothing about views outside D. The conclusion is pointwise over $A \in \mathit{chain}$ under a universal quantifier, not inductive over an evolving state. For each amendment A, the proof case-splits on the activeAt conditional. Outside the window, $\mathsf{activeAt}(A, t) = \delta.\mathit{old}$, and the conclusion is the post-expiry hypothesis $\mathsf{BackwardRefines}(\delta.\mathit{old}, \mathit{baseline}, D)$ applied directly. Inside the window, $\mathsf{activeAt}(A, t) = \delta.\mathit{new}$, and the conclusion composes the per-step witness $\mathsf{BackwardRefines}(\delta.\mathit{new}, \delta.\mathit{old}, D)$ with the post-expiry hypothesis: a view in D admitted by $\delta.\mathit{new}$ is admitted by $\delta.\mathit{old}$ and hence by the baseline. Both arms consume the per-step witnesses as data rather than discharging by reflexivity, so each amendment in the chain contributes substantive content.

The shape is the response-side counterpart of the parent paper's `essential_preserved_chain`, which composes per-step essential- admission witnesses over an amendment sequence. The parent theorem ranges over receipts and a chain of constitutions and is structurally inductive over the chain; this theorem ranges over instants and a chain of TTL-bounded amendments and discharges pointwise per amendment with a case split on the TTL window. The witness-threading pattern is shared: each step in either chain contributes a per-step refinement witness that the discharge consumes as data.

Rollback admissibility. The supporting reduction `rollback_admission_composes_with_refinement` states that a rollback receipt admissible under the post-amendment syntactic constitution remains admissible under the pre-amendment syntactic constitution given the global syntactic refinement witness over admits, where rb is the rollback receipt's admission view:

$$\begin{aligned}&\mathsf{admits}(c_{\text{new}}, rb) = \text{true} \ \wedge \ \big(\forall v,\, \mathsf{admits}(c_{\text{new}}, v) = \text{true} \Rightarrow \mathsf{admits}(c_{\text{old}}, v) = \text{true}\big)\\ &\quad \Rightarrow \ \mathsf{admits}(c_{\text{old}}, rb) = \text{true}.\end{aligned}$$

The proof is a direct application of the refinement hypothesis to the rollback receipt's admission. The reduction supplies the rollback- preservation step the headline composition leans on: a rollback receipt produced inside an amendment's TTL window remains admittable when the window expires and the baseline returns.

The two theorems range over the same objects. Both state admission of an AdmissionView under a SyntacticConstitution, decided by admits. What differs is the scope of the refinement premise. The composition theorem consumes the domain-scoped $\mathsf{BackwardRefines}(c_{\text{new}}, c_{\text{old}}, D)$, which quantifies only over views in D and is the witness a checked ConstitutionalDelta carries; the rollback reduction consumes the unrestricted GlobalSynBackwardRefines, which quantifies over every view. Composing the two therefore carries a side condition: the rollback receipt's admission view must lie in the delta's domain D, since outside D the composition theorem asserts nothing.

The closure representation the substrate retains under Chio.Treaty.Legacy is related to the syntactic one pointwise: the closure induced by a syntactic constitution admits a view exactly when admits does. That is the parent substrate's bridge_pointwise, restated in the paper-local file as `closure_to_syntactic_admission_bridge` so that both reductions can be read against either representation. It is an instance of the parent theorem, not a new proof.

Bilateral destructive admission. The bilateral reduction `destructive_action_requires_bilateral_admission` specializes the parent paper's `treaty_admission_iff_predicate_intersection` to the destructive subclass of the action taxonomy. For a bilateral envelope binding one admission view to a destructive action class under a bilateral treaty, admission holds iff both the device polity's and the operator polity's predicates accept that view. The reduction is structural rather than novel: the parent theorem already establishes the bilateral intersection; this theorem renames it for the destructive class. The contribution at this point is the typed envelope, not a new theorem.

Threat model. The adversary can author and sign requests, attempt to skip the rollback receipt, forge or delete the rollback receipt in the local ledger, backdate or freeze the substrate counter, omit the acknowledgement receipt, present a rollback receipt for an execution it did not perform, or claim that an inverse executor completed when it did not. The adversary cannot break Ed25519, SHA-256 collision resistance, or canonical JSON correctness. The substrate counter itself is an assumption: a compromised counter falsifies the TTL claim in the same way a compromised signing key falsifies admission. The counter is named as a foundational assumption in Section 9.

The model discharges two obligations. First, the typed executive- action constructor refuses to build a witness without a positive TTL and a class tag, so the type system rules out unbounded acts on the construction side. Second, the composition theorem pins the chain's trajectory to the baseline at every instant, so an amendment ratchet that aggregates predicate drift through a sequence of TTL-bounded amendments cannot accumulate without violating a per-step refinement witness. The third obligation, namely that the rollback receipt faithfully witnesses the inverse executor's completion, is a runtime obligation discharged by the executor's signed completion against the forward effect's content hash and is not a theorem in the bounded model.

5 Implementation

This checked-in artifact does not ship an endpoint enforcement engine. It specifies the implementation boundary an endpoint engine must satisfy: twelve action variants, four reversible variants with inverse executors, content-hash-gated rollback constructors, and a four-receipt grammar. The substrate inherits the canonical-byte signing path, ledger discipline, and verifier-owned trust store conventions from the parent paper. Claims about response executors and proof packages below are implementation obligations until their code lands in this repository.

Action plan and execution. The response plan is bound at request time: `EndpointResponsePlan` carries action identifier, action class, dry-run flag, root node identifier, graph slice identifier, TTL in seconds, rollback reference handle, reason, issued-at and expires-at timestamps, and node and edge counts. The expires-at timestamp is derived structurally from issued-at and TTL. The TTL validator rejects zero and out-of-range durations at admission; the upper bound is the deployment-instance constant `EDR_MAX_RESPONSE_TTL_SECONDS`.

The forward executor dispatches on the action class. File quarantine is a hashed read followed by `fs::rename` into the quarantine root. Persistence disable is the same shape with a different destination subtree under the same quarantine root. Process-tree suspend issues `libc::SIGSTOP` to each pid in the target list in reverse-order traversal of the process tree. Egress restriction writes a policy snapshot file at the network-extension policy path and invokes the network-extension reload entry point. Each executor returns a signed execution receipt that records the forward effect set; an effect carries effect identifier, effect type, target, artifact path, content hash, and byte count.

Rollback executor. The rollback executor's inverse pair runs on the four reversible variants. File quarantine's inverse reads the artifact, hashes it, verifies content-hash parity against the recorded forward hash, refuses the operation if the original target path now exists, and issues the inverse `fs::rename`. Persistence disable's inverse has the same shape. Process-tree suspend's inverse issues `libc::SIGCONT` to each pid; the substrate records which pids in the original set have since exited and reports the partial-inverse status. Egress restriction's inverse deactivates the egress ledger entry and re-syncs the network-extension policy. Each inverse executor emits a rollback receipt (`EndpointResponseRollbackReport`) that references the execution identifier and the action identifier, and the receipt is appended to the rollback ledger before the next dispatch.

Acknowledgement loop. The acknowledgement receipt (`EndpointResponseAcknowledgementReport`) is emitted by an endpoint-initiated postback to the control plane. The acknowledgement identifier, execution identifier, action identifier, acknowledged-by key, and optional control-correlation token are bound to the receipt. The substrate does not mutate execution-ledger state on acknowledgement; the acknowledgement is an additive receipt event the verifier-owned correlator ingests.

Receipt signing and verification. Each response receipt is signed by the endpoint's Ed25519 keypair over the canonical-JSON body. The signing path is shared across the four receipt families, which is the substrate's chosen trust granularity for the response engine. Bilateral cosignature on the destructive subclass is a substrate-level obligation discussed in Section 9: the deployment instance carries the single-signer path; the bilateral path is a substrate primitive inherited from the parent paper's `crates/trust/chio-federation/src/bilateral_dsse.rs` that the response engine is not wired against.

Table 1: Measurement status. Withheld rows have a target surface but no measurement harness within this evaluation; the gap is named in Section 9.

Metric	Path	Status
Forward executor latency, four reversible variants	rename, signal, policy reload	planned; executor harness absent
Rollback executor latency, four reversible variants	inverse of the above	planned; executor harness absent
Acknowledgement round-trip	control-plane postback	planned; proof endpoint absent
Replay-corpus admission stability across the four-receipt grammar	fixture corpus	planned; corpus absent
Destructive-variant admission latency (bilateral cosignature)	treaty intersection	withheld; not wired
Sensor-to-decision latency	Endpoint Security subscription	withheld; sensor stub
TTL auto-expiry latency	background scheduler	withheld; scheduler absent

Proof bundle. No checked-in proof endpoint backs this paper yet. The required proof bundle shape is: execution record, graph slice, provider and sensor state, and six receipt families (request, execution, evidence-bundle-manifest, lifecycle transitions, rollback, acknowledgement). A future endpoint `GET/api/v1/agent/edr/response-executions/{id}/proof` must return that bundle and let the verifier replay canonical bytes against the polity's pinned public keys while asserting actor continuity across the chain. Until then, the proof bundle is a design contract, not a deployed implementation claim.

6 Evaluation

The evaluation section records what a deployment evaluation must measure. This repository does not yet ship the response executors, proof endpoint, or replay corpus needed to report those measurements as results. Absent measurements are marked rather than estimated.

The figures reported here are point measurements taken on a single development workstation (Apple Silicon, macOS 14.x, Darwin 23.x kernel) rather than statistically-confident measurements over N runs with reported confidence intervals. The substrate's executor structure and the canonical-byte signing path are reproducible on any host with the same Rust toolchain and a Unix-family filesystem, but the latency distributions are workstation-specific and the kernel-side cost terms (`fs::rename`, signal delivery, network-extension reload) are sensitive to the host's filesystem class, scheduler tuning, and extension load. A defensible USENIX-grade evaluation requires fresh runs against a documented hardware matrix with $N \geq 30$ per cell, a stated warm-up policy, and confidence intervals on each latency cell; that protocol is named here as a deployment requirement and the present numbers are honest order-of-magnitude statements rather than publishable cells.

Forward and rollback latency. The four reversible variants share an executor structure. File quarantine and persistence disable measure as a single `fs::rename` per target plus the canonical-JSON signing path; the rename is microseconds on a local filesystem and the signing path is dominated by canonical-JSON serialization rather than by Ed25519. Process-tree suspend measures as `libc::kill` delivery to each pid in the target list; latency is constant per pid up to the scheduler's

interrupt delivery time. Egress restriction measures as a policy-file write followed by the network-extension reload request; the reload is the dominant term and varies with the extension's internal reload path. The rollback variant of each pair measures the inverse executor against the same shape, with the content-hash gate contributing a single hash recomputation.

The receipt append is not atomic with the side effect. The forward executor invokes the OS primitive (`fs::rename`, `libc::kill`, policy reload) before appending the execution receipt to the ledger; a crash in the window between effect and append leaves a live side effect with no execution record, weakening the "rollback closes the chain" property for that window. The race is named in Section 9 and closed by a write-ahead ledger redesign that appends a pending record before the effect and marks it succeeded after.

Acknowledgement round-trip. The acknowledgement round-trip measures from execution-receipt completion to acknowledgement-receipt completion. The path includes the endpoint's signing of the acknowledgement body and the control-plane postback. The signing path is shared with the other receipt families; the postback latency is dominated by network round-trip and the control-plane handler.

Replay-corpus stability. The replay corpus is not shipped in this repository. The required corpus must contain fixture-backed action chains exercising the four-receipt grammar for each reversible variant. Each fixture should be a deterministic action chain whose admission decisions are canonical-byte stable across machines. The intended stability property is the reproducibility of canonical verdicts rather than re-execution of the OS-side executors. The corpus must distinguish positive chains (request, execution, rollback, acknowledgement all admitted) and negative chains (rollback missing, acknowledgement missing, content-hash mismatch, replay attack on the execution identifier) before this row can be reported.

Destructive subclass. The destructive subclass is reported as a deployment gap rather than a measurement. The deployment instance's destructive executors run under the endpoint's single Ed25519 keypair; the substrate-level bilateral cosignature path is not wired into the response engine. The empirical claim about the destructive subclass is therefore the paper-local reduction `destructive_action_requires_bilateral_admission`, an instance of the substrate's `treaty_admission_iff_predicate_intersection`, not a deployed measurement. Wiring the bilateral path requires an operator-polity key custody design that is outside the scope of the present construction.

Sensor and scheduler gaps. The endpoint sensor subscription is a stub on macOS Endpoint Security: the entitlement is declared, the class is structurally present, but `es_new_client` is not called and no event stream is ingested. The sensor-to-decision latency is therefore not measurable on the current deployment instance. The TTL auto-expiry scheduler is similarly absent: the `/expire` HTTP endpoint exists but no background task invokes it on a counter advance. Both gaps are named in Section 9.

Reproducibility. Measurements are deployment-instance-local; generalization requires fresh runs on stated hardware. The fixture corpus is canonical-byte reproducible across machines; the latency numbers are not. The signing path uses canonical JSON over RFC 8785 conventions and Ed25519 signatures; canonical-byte equivalence across machines is the same property the parent paper's replay corpus asserts.

7 Discussion

The reversible-action discipline extends the amendment-side type-level rule of the prior substrate [6] to the response side, for the class of actions whose effects are reversible by an inverse executor. The interpretation of the extension is constrained by what the substrate proves and what it does not. Three threads of discussion follow: the relationship to constitutional emergency powers, the relationship to agentic-AI oversight, and the relationship to the substrate's notion of time.

Emergency powers and bounded action. A natural framing for time-bounded constitutional change is the literature on emergency powers, particularly the post-9/11 work on deliberative versus emergency authority. The framing is partially apt: a TTL-bounded amendment looks like an emergency exception, and the auto-reversion at TTL expiry looks like a sunset clause. The framing is also partially misleading: real emergency powers are notable for the absence of reliable rollback. A government that invokes emergency authority does not in practice undo the actions taken during the emergency; the very property the TTL discipline formalizes is the property emergency-powers regimes have struggled to guarantee. The substrate therefore does not claim to formalize emergency powers; it formalizes a stricter discipline (timed reversal with type-level invariant) that lies inside the space emergency- powers scholarship surveys.

The deeper point is that the discipline is conservative. The prior substrate's amendment refinement obligation [6] already requires backward refinement: an amendment cannot widen admission on receipts already admitted. The TTL-bounded extension keeps the refinement obligation per step and adds the reversion. A constitutional change that genuinely cannot be reverted (debt restructuring, retroactive statute, founder override, treaty repudiation) does not fit the TTL-bounded class; the prior substrate records such events as crisis artifacts that break refinement and are signed as such. The TTL-bounded class is a strict subclass of the prior substrate's amendment space.

Agentic-AI oversight. An autonomous agent taking a destructive action without a paired rollback is exactly the bounded-action failure mode the substrate forbids at the type level. A typed substrate that refuses to construct an executive action without a TTL witness and an action class tag is a primitive other safety systems can cite without adopting the substrate as a whole. Constitutional AI conditions outputs at training time; capability evaluations bound what a model is likely to do in advance; the substrate here gates whether a specific cross-boundary action enters the receipt log without the counterparty's signature.

The bilateral-cosignature reduction on the destructive subclass is the natural primitive for human-in-the-loop deployment of agentic systems: a destructive action authored by an agent admits only when the deploying organization's polity also signs. The reduction does not solve the operational-discipline problem already named by the prior substrate [6] (a single actor operating both keys collapses the bilateral envelope to one-of-one), but it gives the discipline a typed obligation rather than a policy file. The Anthropic alignment-evaluation work on alignment-faking and situational-awareness is the upstream motivation; the reversible-action substrate is the downstream discipline.

Time and the substrate. The substrate treats time as a monotonic integer counter. The TTL invariant is a duration in counter units, and the auto-reversion at TTL expiry is a substrate-level event triggered by the counter advancing past the expiry instant. The substrate model abstracts the real-time scheduler that advances the counter; the scheduler is a deployment-side obligation. The implication for the threat model is that a compromised counter falsifies the TTL claim in the same

way a compromised signing key falsifies admission. The substrate does not trust the endpoint's wall clock; it trusts the counter's monotonicity and the scheduler's faithfulness in advancing it.

The substrate's notion of partition (the partition-contingency mode on the trust ladder, carried by the prior substrate [6]) introduces a second relationship to time. A partition is a window during which the substrate cannot exchange receipts with peer polities; the substrate records the partition's start and end as receipts and conditions admission during the partition on the ladder's partition-contingency rules. A TTL-bounded amendment whose TTL spans a partition is a substrate-level case the model does not yet cover: the auto-reversion at TTL expiry should fire even if the substrate is partitioned, but the reversion's receipt cannot be exchanged with peers until the partition ends. The substrate model treats this as a deployment-side obligation rather than as a substrate-level theorem.

What the substrate does not extend. The reversible-action discipline does not extend the type-level rule to actions whose effects are not reversible by an inverse executor. Process-tree terminate, network isolate, and grant revoke are destructive: the substrate's response to these is to require bilateral cosignature at admission, not to record a rollback. The substrate makes the distinction between reversible and destructive explicit at the type level, but it does not turn a destructive action into a reversible one. The category boundary is empirical (existence of an inverse executor) rather than constitutional.

8 Related Work

The reversible-action substrate is bounded by five literatures: transaction processing and the saga pattern, capability revocation in operating systems, endpoint detection and response engineering, the reversible-computing line, and the verified-systems pattern.

Compensating transactions and sagas. The saga pattern, introduced for long-running transactions whose commit boundary is weaker than serializable schedules, pairs each forward step with a compensating step that semantically undoes the forward step [1]. Workflow nets extend the model with timed arcs and soundness criteria deciding whether every reachable marking has a path to a final marking. The reversible-action substrate inherits the saga vocabulary on the four reversible action variants: the inverse executor is the compensating step, and the rollback receipt is the typed artifact recording its completion. The substrate does not inherit the workflow-net machinery; it has no timed transition system beyond the monotonic counter, and the soundness criterion the workflow-net literature proves is a strict generalization the substrate does not claim. The contribution relative to sagas is the constitutional-admission layer above the compensating step: a rollback receipt is admissible only when the polity's predicates accept its canonical body, which the saga literature treats as a workflow-engine concern rather than as a polity-level decision.

Capability revocation in operating systems. EROS, seL4, and Capsicum treat capability revocation as a typed operation on a capability token. The capability becomes unusable after revocation, and the revocation event is a checkable artifact at the machine boundary. The reversible-action substrate lifts the same discipline to the cross-organizational boundary where the receipt graph crosses kernels: a revoke-grant action is destructive, requires bilateral cosignature, and the constitutional admission predicate gates the revocation. The OS-capability literature solves revocation at one kernel; the substrate gates revocation at the receipt-graph boundary between kernels.

Endpoint detection and response. Vendor EDR engines have shipped TTL fields and manual rollback workflows for a decade. CrowdStrike Falcon, SentinelOne, Microsoft Defender for Endpoint, and Sublime Security each record an action identifier, target host, action class, TTL duration, and rollback handle for response actions; auditors inspect the rollback handle by hand when something needs to be reverted [4]. The typed four-receipt grammar (request, execution, rollback, acknowledgement) is a normalization of existing vendor schemas; the constitutional admission predicate on the rollback receipt is the substrate-level addition. The substrate is not a new EDR; it is a typed discipline the existing engines could in principle adopt.

Provenance and lineage. Why-provenance, where-provenance, and how-provenance distinctions in the database literature [2] are structurally related to the receipt graph's lineage relations. A rollback receipt is a how- provenance fragment: it records the inverse executor's signed assertion that the OS operation completed, and the receipt graph records the lineage from request to execution to rollback. The substrate does not extend the provenance literature; it specializes it to the cross-kernel-admission case the prior substrate [6] already named.

Reversible computing. Reversible computing originates with Bennett's logical-reversibility construction [7], which shows that a Turing machine can be made information-preserving by carrying a history tape so that every forward step has a unique inverse. The Pendulum line realized the discipline in hardware [8]: a reversible-logic processor whose instruction set is closed under inversion and whose thermodynamic cost per operation is bounded by Landauer's principle rather than by the clock. The Janus programming language [9] formalizes the same discipline at the source-language level: every statement has a syntactic inverse and the language admits an invertible self-interpreter. The reversibility in this literature is thermodynamic-physical: each forward step is recoverable because the runtime preserves the information needed to undo it. The reversibility carried by a reversible-action substrate is structural-policy: each forward step is admissible at the polity's predicates only when paired with a typed rollback witness that references the original execution receipt and the inverse executor's signed completion. The two senses share the word and the structural shape (paired forward and inverse, history preserved as data), but the substrate makes no thermodynamic claim and inherits no Landauer bound.

Verified systems. The Lean-based industrial verification pattern is established by AWS Cedar (Lean specification plus differential-tested Rust runtime) and SampCert (Lean-verified differential-privacy primitives deployed inside AWS Clean Rooms) [3]. The older systems- verification line is the foundational reference set. CompCert [10] established that a realistic optimizing C compiler can be verified end-to-end against a formal semantics; seL4 [11] extended the discipline to an operating-system kernel, with the abstract specification, the C implementation, and the binary all verified against each other; IronFleet [12] extended it to a distributed state-machine-replicated service. These verified-systems efforts establish that formal verification at the artifact level is feasible for production-scale code. The contribution of the prior substrate [6] and of the reversible-action extension is at the substrate-policy level rather than the artifact level: the Lean theorems condition the admission predicates and the amendment refinement obligations on which the runtime depends, not the runtime's compiled code itself. The treaty intersection and amendment refinement theorems extend the policy-verification pattern to cross-kernel admission; the TTL-bounded amendment chain theorem added here completes the shape on the response side.

Table 2: Assumption ledger for the response-side construction. The parent paper's ledger applies in full; the rows below name the additional substrate-level obligations.

Assumption	Used for	Residual risk
Substrate counter is monotonic and faithfully advanced by a real-time scheduler.	TTL expiry, auto-reversion, partition-window bounds.	A compromised counter falsifies the TTL claim. The substrate does not trust the endpoint's wall clock; it trusts the counter's monotonicity.
Inverse executors are content-hash-checked against the forward effect's recorded artifact.	Rollback receipt admissibility.	A forged artifact that matches the forward hash collapses the inverse check.
Ledger append happens before the side effect's external visibility.	Crash safety on the rollback path.	The deployment instance appends the execution receipt AFTER the side effect runs; a crash between effect and append leaves a live side effect with no execution record.
Bilateral cosignature on the destructive subclass is drawn from independent principals.	The bilateral reduction.	Inherited from the parent paper; collapses to one-of-one when a single actor operates both keys.

Agentic-AI safety. The alignment-evaluation literature (METR, UK AI Safety Institute Inspect, ARC Evals) [5] bounds what a model is likely to do during pre-deployment evaluation; the reversible-action substrate gates whether a deployed agent's destructive action crosses the trust boundary at runtime. The two disciplines compose rather than compete: an evaluation result is an input predicate to the polity's constitution, and the constitution's response to a destructive action includes the bilateral-cosignature reduction. Constitutional AI conditions outputs at training; the substrate gates outputs at deployment.

9 Limitations and Threats to Validity

The construction inherits the parent paper's limitations and adds limitations specific to the response side. The assumption ledger below is restricted to the new obligations; the parent paper's ledger covers the cryptographic, library, operational, and legal assumptions common to both constructions.

The remaining limits are:

- **TTL auto-expiry scheduler.** The substrate model assumes a scheduler advances the counter and triggers expiry callbacks. The deployment instance exposes the `/expire` HTTP endpoint but no background task invokes it. The headline composition theorem is unfalsifiable against the current binary without the scheduler; "terminates within its TTL" is operator-dependent rather than type-level.
- **Missing inverse executors on the destructive subclass.** Process-tree terminate, grant revoke, and evidence collection are destructive: each has a forward executor but no inverse. The bilateral reduction applies as an admission obligation; an empirical measurement of the destructive chain requires inverse executors that do not exist.
- **Crash safety on the ledger append.** The deployment instance runs the OS executor before the ledger append. A crash between effect and append leaves a live side effect with no

execution record. A write-ahead ledger pattern (append a pending record before the effect, then mark succeeded after) would close this; the substrate model and the headline theorem do not require it, but the empirical chapter's "rollback closes the chain" claim is weakened by the crash window.

- **Bilateral cosignature not wired into the response engine.** The substrate's bilateral DSSE verifier exists; the response engine signs every receipt under a single endpoint keypair. The destructive subclass's bilateral admission is therefore a substrate-level obligation, not a deployed measurement. Wiring requires an operator-polity key custody design.

- **Single-actor bilateral collapse.** Inherited from the parent paper. Two keys held by a single actor satisfy two-key DSSE but not party-independence; the destructive admission reduces to one-of-one signatures. The construction here narrows the consequence (the destructive class becomes operator-key- dependent) without solving the underlying problem.

- **Sensor stub on macOS Endpoint Security.** The deployment instance declares the entitlement but does not subscribe via `es_new_client`; sensor-state attestation on response receipts is bounded to the Network Extension verdict path, the tool-preflight admission hook, and the package-manager runtime guard. The substrate model treats sensor state as an audited input; the deployment instance's input fidelity on the Endpoint Security path is below the substrate model's assumption.

- **Sensor-state fidelity on rollback and acknowledgement receipts.** The deployment instance attaches a placeholder sensor state on rollback and acknowledgement emitters rather than a fresh snapshot; a constitution that requires sensor-attested rollbacks sees synthetic data. The substrate model treats this as a deployment refinement obligation.

- **Rollback receipts under concurrent invocation.** The deployment instance has no per-execution rollback lock; manual rollback and TTL-driven rollback can fire concurrently. The egress ledger mutex serializes egress rollbacks; the file-rename rollbacks rely on filesystem-level atomicity of `rename(2)` plus a TOCTOU-racy existence check. The substrate model does not address concurrency.

- **Chain-of-rollback semantics.** A rollback receipt inverting another rollback receipt is not modeled. The `RollbackReceipt` structure has a single `rollbackOf` field; an inductive chain requires a structure the substrate does not introduce.

- **Bounded paper model.** The model encodes the TTL- bounded amendment chain composition theorem and four supporting bridges; it does not verify the whole deployment instance. The model is paper-local: it is checked by hand against the substrate's syntactic treaty model and is not in the substrate's proof manifest or theorem inventory. The deployment instance discharges the substrate-level obligations by code citation rather than by mechanized refinement.

- **rfl-gate on the headline theorem.** The headline composition theorem `ttl_bounded_amendment_chain_preserves_baseline` does not discharge to *rfl*: the paper-local mechanization proves it pointwise per amendment by a case split on the TTL window, composing the two refinement witnesses on the explicit finite admission domain in the in-window arm. The result is finite-domain: it holds on the domain the witnesses were checked against and says nothing about admission views outside it.

- **Operator-key custody at scale.** The bilateral admission on the destructive subclass requires an operator-polity key whose custody, rotation, revocation, and compromise recovery is outside the present construction. The parent paper's inheritance applies: per-kernel Ed25519 signing assumes a key- management story (HSM, KMS, TEE-bound key handles) the present construction does not specify.
- **Override path for false-rollback admission.** The substrate denies a rollback receipt whose canonical body fails the polity's predicates. The construction does not define which capability authorizes an admit-under-protest rollback, what crisis artifact the override produces, or how the override is audited. The same gap applies to the parent paper's override authority limitation.
- **Real-time semantics.** The substrate treats time as a monotonic counter; partition behavior across a TTL boundary is not modeled. The auto-reversion at TTL expiry during a partition is a deployment-side obligation rather than a substrate-level theorem.

10 Conclusion

The amendment side of constitutional admission is type-conditioned on a backward-refinement witness; the response side, for actions whose effects are reversible by an inverse executor, can be conditioned on a positive TTL witness and an optional rollback receipt that closes the action chain. The composition theorem `ttl_bounded_amendment_chain_preserves_baseline` relates the response side to the amendment side: a chain of TTL-bounded amendments, each carrying a per-step refinement witness, preserves the baseline constitution at every instant under the auto-reversion semantics of the TTL window. The reduction `destructive_action_requires_bilateral_admission` specializes the treaty intersection theorem of the prior substrate [6] to the destructive subclass of the action taxonomy, making human-in-the-loop oversight of destructive enforcement a typed obligation rather than a policy file. The endpoint enforcement engine grounds the substrate in four reversible action variants with content-hash-gated inverse executors and a four-receipt grammar; the TTL auto-expiry scheduler, the missing inverse executors on the destructive subclass, and the bilateral cosignature path remain substrate-level obligations the runtime does not yet deploy. The headline composition theorem discharges in the paper-local mechanization by a case split on the TTL window with per-step witnesses on an explicit finite admission domain, not by reflexivity; on that domain the response side carries the same finite-domain refinement discipline as the amendment side of the prior substrate. Promoting the mechanization into the substrate's proof root is the next step.

References

[1] H. Garcia-Molina and K. Salem. *Sagas.* Proceedings of the 1987 ACM SIGMOD International Conference on Management of Data, pages 249–259, 1987.

[2] J. Cheney, L. Chiticariu, and W.-C. Tan. *Provenance in databases: Why, how, and where.* Foundations and Trends in Databases, 1(4):379–474, 2009.

[3] J. W. Cutler, C. Disselkoen, A. Eline, S. He, K. Headley, M. Hicks, K. Hietala, E. Ioannidis, J. Kastner, A. Mamat, D. McAdams, M. McCutchen, N. Rungta, E. Torlak, and A. Wells. *Cedar: A new language for expressive, fast, safe, and analyzable authorization.* Proceedings of the ACM on Programming Languages, 8(OOPSLA1):670–697, 2024.

[4] Vendor endpoint-detection-and-response response-action schemas (CrowdStrike Falcon, SentinelOne, Microsoft Defender for Endpoint, Sublime Security). Placeholder pending pinned product-documentation references.

[5] Alignment-evaluation harnesses and task suites (METR, UK AI Safety Institute Inspect, ARC Evals). Placeholder pending pinned references to the venues of record.

[6] Anonymous. *Receiver-Owned Bilateral Admission for Cross-Organization Agent Tool Calls.* Submitted for review.

[7] C. H. Bennett. *Logical reversibility of computation.* IBM Journal of Research and Development, 17(6):525–532, 1973.

[8] C. J. Vieri. *Reversible computer engineering and architecture.* Ph.D. dissertation, Massachusetts Institute of Technology, 1999.

[9] T. Yokoyama and R. Glück. *A reversible programming language and its invertible self-interpreter.* Proceedings of the 2007 ACM SIGPLAN Workshop on Partial Evaluation and Program Manipulation (PEPM), pages 144–153, 2007.

[10] X. Leroy. *Formal verification of a realistic compiler.* Communications of the ACM, 52(7):107–115, 2009.

[11] G. Klein, K. Elphinstone, G. Heiser, J. Andronick, D. Cock, P. Derrin, D. Elkaduwe, K. Engelhardt, R. Kolanski, M. Norrish, T. Sewell, H. Tuch, and S. Winwood. *seL4: Formal verification of an OS kernel.* Proceedings of the 22nd ACM Symposium on Operating Systems Principles (SOSP), pages 207–220, 2009.

[12] C. Hawblitzel, J. Howell, M. Kapritsos, J. R. Lorch, B. Parno, M. L. Roberts, S. Setty, and B. Zill. *IronFleet: Proving practical distributed systems correct.* Proceedings of the 25th ACM Symposium on Operating Systems Principles (SOSP), pages 1–17, 2015.